DOCUMENTACIÓN DE PROYECTO

Monitor MikroTik

Herramienta de gestión, configuración y monitoreo de redes MikroTik basadas en OSPF

Autor: Christian

Área: Redes y Automatización

Versión documentada: 1.0

Fecha: Octubre 2026

Del descubrimiento automático de la topología al monitoreo en vivo con reconfiguración inteligente de rutas.

Índice

1. Resumen
2. Introducción y planteamiento del problema
3. Objetivos
4. Marco teórico y conceptos previos
5. Metodología: desarrollo paso a paso
6. Cómo funciona internamente (explicación didáctica)
7. Resultados y validación
8. Conclusiones
9. Trabajo futuro
10. Glosario
11. Referencias

1. Resumen

Monitor MikroTik es una aplicación de escritorio que permite administrar una red de routers MikroTik (RouterOS) de forma casi completamente automática. A partir de una búsqueda en la red local, descubre los equipos, se conecta a ellos, detecta cómo están cableados entre sí y configura el protocolo de enrutamiento dinámico **OSPF** sin que el operador tenga que indicar manualmente ninguna dirección IP ni ninguna red.

Además de configurar, la aplicación *monitorea en vivo*: mide el tráfico real de cada enlace, dibuja la topología, genera alarmas cuando un enlace se satura o se cae y, ante ciertas condiciones, *redirige el tráfico por caminos alternativos* cambiando los costos de OSPF de manera automática y programada.

> **Idea central:** la red se describe a sí misma. El operador enciende los equipos y la herramienta hace el resto: descubrir, conectar, configurar, vigilar y reaccionar.

2. Introducción y planteamiento del problema

2.1 Contexto

Configurar una red de varios routers a mano es lento y propenso a errores: hay que asignar direcciones a cada enlace, definir identificadores de router, declarar las redes que participan del enrutamiento y verificar, uno por uno, que cada enlace haya quedado bien. Cuando la red cambia (se agrega un cable, se cambia una placa de red, se cae un equipo) hay que repetir buena parte del trabajo.

2.2 Problema

El trabajo manual plantea tres dificultades concretas:

- **Coordinación:** las dos puntas de un cable deben usar direcciones compatibles de la misma subred; si una se equivoca, el enlace no levanta.
- **Escalabilidad:** cuanto más grande la red, más tedioso y frágil es mantenerla a mano.
- **Operación:** una vez configurada, no hay visibilidad permanente del estado real (tráfico, caídas, saturación) ni capacidad de reaccionar automáticamente ante fallas.

2.3 Solución propuesta

Una aplicación que automatiza el ciclo completo de vida de la red: **descubrir → conectar → configurar → vigilar → reaccionar**. Cada una de estas etapas se resuelve con una técnica concreta que se explica en este documento, tanto en su parte conceptual como en su uso práctico paso a paso.



3. Objetivos

3.1 Objetivo general

Desarrollar una herramienta que simplifique la puesta en marcha, la configuración y el monitoreo continuo de una red MikroTik, reduciendo al mínimo la intervención manual del operador.

3.2 Objetivos específicos

- Detectar automáticamente los routers MikroTik presentes en la red local.
- Establecer sesiones simultáneas con varios equipos mediante la API de RouterOS.
- Descubrir la topología real (qué equipo está conectado a cuál y por qué puerto).

- Generar y aplicar la configuración de OSPF sin datos cargados a mano.
- Medir y visualizar el tráfico de cada enlace en tiempo real.
- Alertar y reaccionar ante saturación o caídas, redirigiendo el tráfico cuando corresponda.
- Permitir guardar, comparar y restaurar configuraciones de los equipos.

4. Marco teórico y conceptos previos

Antes de ver el desarrollo, es útil entender *qué piezas* usa la aplicación y *por qué*. Esta sección presenta los conceptos en lenguaje sencillo.

4.1 MikroTik y RouterOS

MikroTik es una marca de equipos de red y **RouterOS** es su sistema operativo. Los equipos se administran por consola o por distintos servicios de red. Uno de ellos es la **API**, un canal pensado para que programas externos lean y escriban la configuración de forma estructurada.

4.2 La API en el puerto 8728

RouterOS expone su API binaria en el puerto **8728**. La aplicación se comunica con ese puerto usando la librería de Python `librouteros`, que permite ejecutar comandos como si fueran consultas a una base de datos (leer interfaces, rutas, vecinos, etc.) y también modificar la configuración.

4.3 Descubrimiento de vecinos: MNDP / LLDP

Los dispositivos de red pueden "anunciarse" a sus vecinos mediante protocolos de descubrimiento (MNDP en MikroTik, LLDP en el estándar abierto). Cada equipo publica su identidad y la lista de vecinos vistos por cada puerto. La aplicación usa esta información para **deducir el cableado** sin que nadie le diga cómo está armada la red.

4.4 Enrutamiento dinámico y OSPF

OSPF es un protocolo mediante el cual los routers se cuentan automáticamente qué redes conocen y calculan la mejor ruta. Para que funcione, los routers vecinos deben (a) tener una dirección de router única (*router-id*), (b) compartir una red en cada enlace y (c) estar en la misma *área*. La aplicación genera esas tres condiciones sola.

4.5 Enlaces punto a punto con subredes /30

Un enlace entre dos routers solo necesita dos direcciones. La forma más limpia de representarlo es una subred `/30` (cuatro direcciones: red, dos equipos, broadcast). La aplicación asigna direcciones dentro de un `/30` de manera **determinista**: cada extremo llega al mismo resultado sin necesidad de coordinarse con el otro.

4.6 El costo OSPF como forma de direccionar el tráfico

OSPF elige el camino de menor “costo”. El costo se deriva del ancho de banda, pero puede fijarse manualmente. Subir el costo de un enlace lo vuelve menos preferido; la aplicación aprovecha esto para **forzar o evitar caminos** de forma controlada.

4.7 Tecnologías de la aplicación

Componente	Rol en la solución
Python 3.12	Lenguaje de programación de todo el sistema.
Flask	Servidor web que expone la lógica como servicios (`/api/…`).
librouteros	Comunicación con la API de RouterOS (puerto 8728).
pywebview	Ventana de escritorio nativa que muestra la interfaz.
YAML	Formato de los archivos de configuración e inventario.
PyInstaller	Empaqueta todo en un ejecutable `.exe` .

5. Metodología: desarrollo paso a paso

El proyecto se construyó como una secuencia de pasos, cada uno agregando una capacidad sobre la anterior. A continuación se explica *qué* hace cada paso, *cómo* se usa en la práctica y *por qué* fue necesario.

0 Paso cero: preparar el entorno

Sin un entorno de pruebas no se puede validar nada.

Se dispone de equipos MikroTik virtuales (CHR) conectados en una topología de laboratorio que simula una red real. Sobre esa red se prueba todo. La aplicación requiere Python 3.12 y las dependencias indicadas; la instalación se realiza en un entorno virtual:

```
cd ~/"Monitor MikroTik"
python3 -m venv .venv
source .venv/bin/activate
pip install flask librouteros pywebview netaddr pyyaml pyinstaller
```

1 Descubrir los equipos

No hace falta saber las IP: la red se explora sola.

La pantalla inicial ofrece **“Escanear red local”**. La aplicación revisa todas las redes locales de la computadora y prueba, en cada dirección, si el puerto `8728` está abierto. Los equipos que responden se listan con su identidad y versión. Si un equipo no permite el acceso con credenciales por defecto, se marca como que “requiere credenciales”.

Uso: presionar “Escanear”, elegir un equipo (o marcarlo) e ingresar usuario y contraseña. También se puede escribir una IP a mano si el equipo está fuera del segmento local.

2 Conectar y trabajar con varios equipos

La red tiene muchos routers; hay que administrarlos juntos.

Se puede conectar un equipo o varios a la vez. Una vez conectados, aparece un **selector de equipo activo**: todas las consultas y acciones siguientes se aplican al equipo seleccionado, sin necesidad de reconectar.

3 Consultar la configuración y el estado

Antes de cambiar algo, hay que poder verlo.

Botones de consulta muestran, del equipo activo: interfaces, direcciones IP/ARP/DHCP, rutas y firewall, VLANs y bridges, NAT, estado de OSPF y estado general (CPU, memoria, uptime).

4 Generar el inventario OSPF automáticamente

Evitar cargar a mano qué equipo es cuál y cómo está conectado.

Con **“Generar ospf.yml”** la aplicación se conecta a cada router, lee los vecinos MNDP/LLDP puerto por puerto y reconstruye el cableado. Luego:

- Numera los routers como `R1..Rn`.
- Asigna a cada equipo su *router-id* y su red `/32`.
- Crea una red `/24` por cada enlace detectado.
- Muestra una vista previa y permite guardar el archivo `ospf.yml`.

> Resultado: un archivo de inventario que describe toda la red, generado sin que el operador haya escrito ninguna dirección.

5 Levantar OSPF (paso 1)

Que los routers empiecen a intercambiar rutas.

Con un clic (“1) Levantar OSPF desde YAML”), la aplicación recorre el inventario y, en cada equipo:

- Crea la interfaz virtual del *router-id*.
- Configura la instancia OSPF y el área backbone.
- Declara las redes que deben participar del enrutamiento.

La operación es **idempotente**: si la configuración ya está bien, no toca nada y no reinicia OSPF innecesariamente.

6 Levantar las interfaces según los cables (paso 2)

Un enlace sin direcciones IP nunca levantará.

Aquí se resuelve el punto más delicado: dar a cada cable la dirección que le corresponde. La aplicación, por cada equipo:

- Vuelve a leer los vecinos MNDP para saber qué hay de cada lado.
- Busca la red en común del par y le asigna direcciones `/30` deterministas.
- Elimina direcciones que ya no corresponden y regenera las redes OSPF.

Este paso se puede repetir sin problema: si se agregan cables, placas o routers, se vuelve a ejecutar y la red se reacomoda.

7 Monitorear en vivo

La configuración no alcanza: hay que ver cómo se comporta la red.

El panel de monitoreo se refresca cada pocos segundos y presenta cinco pestañas:

Pestaña	Qué muestra
Tráfico	RX/TX en bits por segundo, uso respecto a la capacidad y estado (UP / DOWN / alarma).
Topología	Grafo de nodos y enlaces, con el estado de cada enlace y el camino que usa el tráfico.
Alarmas	Reglas de alerta y su evaluación ciclo a ciclo.
Direccionamiento	Reglas para forzar o evitar caminos, manuales o por horario.
Métricas OSPF	Costo de cada interfaz y capacidad configurada.

8 Definir alarmas con reacción automática

Detectar problemas y actuar sin esperar a una persona.

Una alarma se define con: equipo, interfaz, métrica (RX, TX, total, suma de red o porcentaje de saturación), operador de comparación, umbral, severidad, tiempos de duración y limpieza, y una **acción**. Si la acción es automática, al dispararse la alarma la aplicación cambia el costo del enlace (ambas puntas) para desviar el tráfico; al cesar, restaura el valor anterior.

9 Forzar caminos de forma programada

A veces se necesita que el tráfico use una ruta concreta en cierto horario.

En la pestaña “Direccionamiento” se arma un camino haciendo clic en el mapa (router → enlaces) y se define qué costo poner a la ruta deseada y qué costo poner a la que se quiere evitar. La regla puede aplicarse a mano o por **franja horaria**. La aplicación comienza a preparar el cambio unos segundos antes del horario, para que OSPF ya haya reconvergido cuando la franja empiece.

10 Guardar, comparar y restaurar configuraciones

Poder volver atrás y auditar cambios.

La aplicación exporta la configuración del equipo a un archivo YAML (por bloques), la compara con otra versión resaltando agregados, eliminados y cambios, y permite **restaurar** una configuración anterior, ya sea completa o seleccionando solo algunos cambios.

6. Cómo funciona internamente (explicación didáctica)

Esta sección explica *por qué la magia funciona*. Cada subapartado responde a una pregunta que un usuario se haría al usar la herramienta.

6.1 ¿Cómo calcula el tráfico en bits por segundo?

Los routers no entregan “la velocidad actual”; entregan **contadores acumulados** de bytes recibidos y enviados. La aplicación toma dos lecturas separadas en el tiempo y aplica una regla simple: la diferencia de bytes multiplicada por 8 (para pasar a bits) y dividida por los segundos transcurridos. Si el contador baja (porque la interfaz se reinició) o pasó demasiado tiempo, la cifra se descarta y se reporta cero, para no mostrar valores absurdos.

6.2 ¿Cómo deduce la topología con solo mirar cada router?

Cada router conoce a sus vecinos directos por MNDP/LLDP. La aplicación reúne estas observaciones de todos los equipos y las cruza: si el puerto A del router 1 ve al router 2, y el puerto B del router 2 ve al router 1, existe un cable entre ellos. Combinando esta información con las direcciones y el estado de OSPF, se dibuja el mapa y se identifica qué camino usa el tráfico.

6.3 ¿Cómo evita que las dos puntas de un cable se contradigan?

Es el truco más elegante del sistema. Para asignar direcciones a un enlace no hace falta que los routers se pongan de acuerdo: cada extremo aplica la **misma fórmula** usando el identificador (router-id) de ambos. El de mayor identificador siempre toma una dirección y el de menor la otra. Así, cada lado calcula lo mismo que el otro sin comunicarse. Si hay dos cables paralelos entre el mismo par, cada uno usa su propia subred, ordenados por la dirección física (MAC), que es idéntica vista desde ambos lados.

6.4 ¿Por qué a veces hay que “forzar” el costo de un enlace?

OSPF deduce el costo del ancho de banda, pero los equipos virtuales a veces no informan su velocidad. Además, un enlace que aparece con “velocidad muy alta” puede recibir todo el tráfico aunque no sea lo deseado. En esos casos la aplicación fija un costo manual. Existe un detalle técnico importante: los equipos no siempre permiten modificar una fila “dinámica”; entonces se crea una fila estática equivalente que la reemplaza.

6.5 ¿Qué pasa si el operador cierra la aplicación?

El sistema de reacción ante alarmas y reglas no vive en la ventana, sino en un proceso de fondo. Aun con la ventana cerrada, un hilo interno mantiene el estado correcto de los costos y, si ya no hay ninguna regla

activa, devuelve los enlaces a su valor por defecto para no dejar “caminos pegados”.

6.6 ¿En qué orden se aplican las decisiones?

Cuando varias cosas podrían afectar el mismo enlace, se respeta una prioridad clara:

Alarma activa > Regla vigente > Costo normal > Costo dinámico

6.7 ¿Cómo se restaura sin romper la red?

Al restaurar, la aplicación no borra y reescribe todo. Compara lo deseado con lo que el equipo tiene realmente y aplica solo las diferencias. Las filas se emparejan por su nombre o dirección (no por posición), de modo que agregar o quitar un elemento no confunde al resto. Los datos que cambian solos (hora, contadores, uptime) se ignoran. Finalmente, las reglas de firewall y NAT se reposicionan porque en RouterOS *el orden importa*.

6.8 Arquitectura general

Capa	Archivo	Responsabilidad
Interfaz	templates/index.html	Toda la pantalla que ve el usuario.
Servidor	app.py	Servicios `/api/…`, sesiones y comparación/restauración.
Motor de red	monitoreo.py	Tráfico, topología, alarmas y reconciliación de costos.
Transporte	transporte_mikrotik.py	Escaneo, conexión y configuración de RouterOS.
Escritorio	app_desktop.py	Ventana nativa y diálogos de archivo.

7. Resultados y validación

- La topología completa se descubre y configura **sin datos manuales**.
- Los enlaces OSPF alcanzan el estado *Full* entre los equipos vecinos.
- El tráfico se visualiza en tiempo real con valores coherentes respecto al uso.
- Las alarmas se disparan y limpian según lo configurado, y sus acciones automáticas cambian y restauran los costos.
- Los cambios por horario se anticipan y se revierten correctamente.
- La restauración de configuración aplica únicamente las diferencias esperadas.

8. Conclusiones

El proyecto demuestra que es posible llevar una red de routers MikroTik de un estado “en blanco” a un estado *plenamente operativo y monitoreado* con una intervención humana mínima. La clave está en aprovechar información que los propios equipos ya generan (vecinos MNDP/LLDP, contadores de tráfico) y en diseñar reglas de asignación deterministas que no requieran coordinación entre los routers.

El resultado no solo ahorra tiempo de configuración, sino que agrega valor operativo: visibilidad continua, alertas y la capacidad de reaccionar ante fallas o congestiones de forma automática y programada.

9. Trabajo futuro

- Soporte para redes de mayor tamaño con segmentación por áreas OSPF.
- Notificaciones externas (correo, mensajería) ante alarmas críticas.
- Histórico y gráficos de tráfico a lo largo del tiempo.
- Modo de solo lectura y control de usuarios con permisos.
- Empaquetado multiplataforma además del ejecutable de Windows.

10. Glosario

Término	Significado
RouterOS	Sistema operativo de los equipos MikroTik.
API (8728)	Canal de administración programática de RouterOS.
OSPF	Protocolo de enrutamiento dinámico entre routers.
Router-id	Identificador único de cada router dentro de OSPF.
MNDP / LLDP	Protocolos de descubrimiento de vecinos.
/30, /32, /24	Notaciones de subred (máscara) usadas para direccionar enlaces.
Costo OSPF	Valor que determina la preferencia de un enlace.
Reconvergencia	Tiempo que tarda la red en adaptarse a un cambio de ruta.
Idempotente	Que aplicar la operación muchas veces deja el mismo resultado que aplicarla una vez.
YAML	Formato de archivo legible usado para configuración e inventario.

11. Referencias

- MikroTik. *RouterOS Documentation* — API, OSPF, MNDP, interfaces.
- MikroTik. *Manual de enrutamiento OSPF*.
- Documentación de *librouteros* y *pywebview*.
- Documentación de *Flask* y de *PyInstaller*.
- Documentación del proyecto: `DOCUMENTACION.md` (referencia técnica interna).

> Este documento complementa el archivo técnico del proyecto. Está pensado para su presentación y lectura académica: explica el *qué*, el *cómo* y el *por qué* de cada decisión.